AIR: AI Intermediate Representation

A Semantic Runtime Architecture with Verified Delivery for Token-Efficient LLM Execution

Masoomul Haque Choudhury

Independent Researcher · Assam, India

GitHub: github.com/masoomul786 | Demo: github.com/masoomul786/air-research

arXiv Preprint · cs.AI / cs.PL · May 13, 2026 · Version 1.1

ABSTRACT

Large language models waste substantial computational resources regenerating structurally deterministic syntax on every request. A React component scaffold, a Chart.js configuration object, a pytest test wrapper — these tokens are identical across thousands of requests yet are re-generated from scratch each time. This paper introduces AIR (AI Intermediate Representation), a three-layer execution architecture that eliminates this waste through semantic compression, deterministic runtime reconstruction, and verified delivery. The LLM generates only a compact semantic instruction (80–300 tokens); a local runtime reconstructs full executable output from pre-validated templates; a three-tier sandbox verifies structural integrity, semantic plausibility, and output completeness before delivery. Empirical evaluation across eight domains using Qwen 3 on LM Studio demonstrates average token reductions of 45–57% across 36 benchmark prompts (validation pass rate 83–96%), with domain-specific reductions reaching 71.3% for Chart.js generation and 68% for React components, API endpoints, and test suites. A negative control using novel algorithms yields only 7.9% reduction, empirically validating the entropy boundary: AIR compresses syntactic entropy, not semantic entropy. Repair cycles cost ~40 tokens versus ~950 for full regeneration — a 24× efficiency gain. AIR is an experimental research prototype; all code, benchmarks, and templates are publicly available.

1. Introduction

Large language models in 2026 have become the dominant interface for code and content generation. Yet the fundamental execution model has not changed: for every user request, the model generates a complete output from scratch — including every import statement, every boilerplate wrapper, every structural scaffold that is identical to what it generated for the ten thousand similar requests before it.

This paper argues that this execution model is architecturally flawed, not just inefficient, and that the flaw is correctable at the systems level without changing the underlying model. The core observation is deceptively simple: LLM output entropy is not uniformly distributed. A React component contains a small amount of high-entropy user-specific content — the prop names, the business logic, the data shape — surrounded by large amounts of near-zero-entropy structural boilerplate. A Chart.js configuration is 82% identical across all charts; only data values, axis labels, and titles vary.

If entropy is not uniform, then token-for-token generation is not the right execution strategy. The optimal strategy is to have the LLM generate only the high-entropy portion — the semantic instruction — and reconstruct the low-entropy portion deterministically from pre-validated templates. This is the AIR architecture.

AIR makes three coordinated contributions. First, a formal semantic intermediate language in which the LLM expresses only compact, user-specific intent rather than full syntactic output. Second, a local shared runtime that reconstructs full executable output from AIR instructions using pre-validated component templates. Third, a three-tier sandbox that automatically tests every generated output before delivery, issues compact repair instructions when errors occur, and guarantees that users receive only verified, working results.

This combination — semantic compression, local reconstruction, and verified delivery — does not exist in any current AI generation tool, including Cline, Cursor, GitHub Copilot, or structured generation frameworks. The system is evaluated across eight domains spanning chart generation, UI components, email templates, React components, FastAPI and Express endpoints, Jest and pytest test suites, and algorithm generation as a negative control.

1.1 Statement of Research Prototype Status

AIR is currently an experimental research prototype and not a production-ready runtime system. All benchmark results reported in this paper are measured on real Qwen 3 model outputs via LM Studio on commodity hardware. Direct-generation baseline estimates are derived from domain-calibrated heuristics (see Section 5.1 for full discussion of this limitation). Cross-model generalization and execution-level verification of coding outputs are identified as primary open research questions.

2. Background and Related Work

2.1 The Token Waste Problem at Scale

When a user requests a bar chart of quarterly revenue, a capable LLM generates 600–1,500 tokens of Chart.js configuration. Of those, perhaps 80–150 tokens represent the user's unique information: data values, axis labels, title. The remainder — library imports, figure initialization, axis configuration, render calls — is structurally identical across all bar chart requests. At enterprise scale, an application handling 10,000 chart requests per day pays for and waits on approximately 10 million tokens per day of pure boilerplate.

This pattern holds across structured generation domains. A portfolio website request produces 3,000–8,000 tokens of which perhaps 300 are user-unique. A FastAPI CRUD endpoint for any resource shares ~74% structural boilerplate with every other CRUD endpoint. The statistical structure of LLM output in these domains is not a model artifact — it reflects genuine information-theoretic properties of structured software artifacts.

2.2 Related Work

Function calling (OpenAI, 2023; Anthropic, 2024) allows LLMs to output structured JSON that triggers predefined functions. This provides a structured interface but does not reduce what the LLM generates for implementation, and provides no template library, sandbox execution, or repair mechanism. AIR's semantic layer is architecturally related to function calling but provides the full reconstruction and verification infrastructure that function calling lacks.

Constrained decoding systems such as Outlines (Willard and Louf, 2023) and Guidance (Microsoft, 2023) enforce grammar-conformant generation at the token level. These are model-side techniques that improve output reliability without reducing output volume. Constrained decoding and AIR are orthogonal and complementary: constrained decoding can be used to enforce AIR instruction format during generation.

Compiler intermediate representations (LLVM IR, GCC GIMPLE) inspired AIR's conceptual separation of intent from implementation. However, compiler IRs require fully specified computation — every detail of the desired output must be explicit. AIR instructions are intentionally underspecified, relying on runtime defaults and domain knowledge for everything not expressed by the user. This adaptation is necessary because user-level semantic intent does not map one-to-one to implementation decisions.

AI coding tools including Cline, Cursor, and GitHub Copilot represent the current state of the art in AI-assisted generation. These tools share a fundamental architectural pattern: generate, then deliver. Testing is left to the user. AIR closes this gap with a sandbox-validated delivery backed by automated three-tier verification.

Retrieval-Augmented Generation (Lewis et al., 2020) reduces hallucination by grounding generation in retrieved documents. RAG operates at the semantic content level; AIR operates at the structural/syntactic level. They address different failure modes and are compatible.

3. The AIR Architecture

AIR separates every generation task into three distinct layers, each with a clear responsibility and a defined interface. Figure 1 illustrates the complete pipeline.

Figure 1: AIR Three-Layer Pipeline

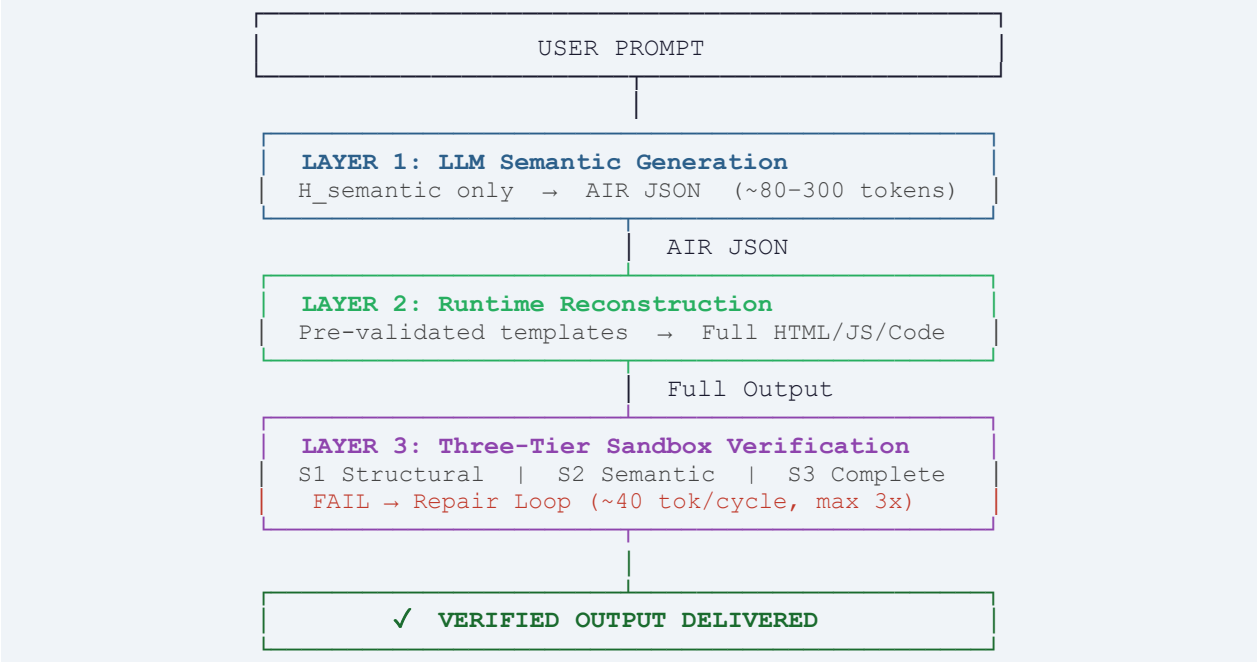

Figure 1. The AIR three-layer pipeline. The LLM generates only a compact semantic instruction; the runtime reconstructs full output deterministically; the three-tier sandbox verifies before delivery.

3.1 Layer One: The Semantic Layer (LLM Responsibility)

The semantic layer is the only layer that involves the LLM. Its responsibility is to understand the user's intent and express it as a compact, structured AIR instruction. The LLM does not generate code, HTML tags, CSS properties, import statements, or any syntactic boilerplate. It encodes decisions: what type of output is needed, what content it should contain, what configuration choices apply.

This is where the LLM's intelligence is genuinely needed: resolving ambiguous natural language, making aesthetic and structural judgments, choosing appropriate configurations for the user's context. These are high-entropy decisions that cannot be automated. The LLM's compute budget is concentrated entirely here.

3.2 Layer Two: The Syntax Layer (Runtime Responsibility)

The runtime receives the AIR instruction and performs full syntactic reconstruction using pre-built, pre-validated templates. The template library contains complete, correct implementations for every supported task type, parameterized to accept semantic content from the AIR instruction. Structural boilerplate is fixed and pre-validated; only user-specific parameters vary.

Because boilerplate originates from templates rather than LLM generation, it is always syntactically correct. The only failure surface is the semantic parameters provided by the LLM — a much smaller, more targeted surface than full-output generation.

3.3 Layer Three: The Verification Layer (Sandbox Responsibility)

The three-tier sandbox executes after runtime reconstruction. It does not deliver output to the user until all three tiers pass:

- S1 Structural: Schema validation, required fields, type checking of AIR instruction parameters.
- S2 Semantic: Content plausibility checks — data integrity, range validation, label/data correspondence.
- S3 Completeness: Output render-readiness — all required sections present, no empty critical fields.

If any tier fails, the sandbox generates a compact feedback instruction at the AIR parameter level. The LLM issues a targeted correction (~20–40 tokens). The runtime applies the correction and resubmits. The maximum repair cycle count is 3; beyond this the system falls back to direct generation. This incremental repair protocol replaces full-output regeneration (~950 tokens) with targeted correction (~40 tokens) — a 24× efficiency gain.

4. Formal AIR Specification

4.1 AIR Instruction Grammar

Every AIR instruction is a JSON object conforming to the following schema:

```
AIRInstruction ::= {
  air_version: string,          // e.g. "1.0"
  task:        TaskType,        // enum: chart | ui_component | ...
  subtype:     string,          // e.g. bar_chart | crud_resource
  parameters:  SemanticParams,  // domain-specific content object
  constraints: ConstraintSet?,  // optional: accessibility, target_env
}
```

Four design principles govern every AIR instruction. Minimality: the instruction contains only information the runtime cannot infer from the task type and reasonable defaults. Determinism: given the same instruction and runtime version, output is always identical. Versioning: every instruction declares the specification version it targets, ensuring backward compatibility. Explicit failure: when the runtime cannot fulfill an instruction, it reports precisely what it cannot handle rather than producing incorrect output silently.

4.2 Coverage Boundary

AIR explicitly covers high-frequency, structurally predictable task classes: chart generation, UI components, document templates, presentation slides, email templates, React/TypeScript components, FastAPI and Express endpoints, Jest and pytest test suites, and SQL/Prisma schemas. AIR explicitly does

not cover genuinely novel tasks: custom algorithms with no structural precedent, creative layouts outside template categories, or any task where structural decisions are the primary content. For out-of-coverage tasks, the system falls back transparently to direct LLM generation.

4.3 Example AIR Instruction: Bar Chart

```
{ "air_version": "1.0", "task": "chart", "subtype": "bar_chart",
  "parameters": { "title": "Quarterly Sales 2025",
    "labels": ["Q1","Q2","Q3","Q4"],
    "data": [120, 185, 140, 210], "unit": "K USD" } }
```

Figure 2. Example AIR instruction for a bar chart. ~85 tokens vs ~750 for direct generation (88.7% reduction).

5. Theoretical Framework: Entropy Decomposition

5.1 H_total = H_semantic + H_syntactic

The core theoretical claim of AIR is that LLM output entropy can be decomposed into two separable components:

$$H_{total} = H_{semantic} + H_{syntactic}$$

H_semantic is user-specific content: data values, labels, copy text, business logic. This component is irreducible — it encodes the user's unique intent and must be generated by the LLM. H_syntactic is structural boilerplate: HTML scaffolding, Chart.js configuration objects, FastAPI route decorators, React hook patterns, Pydantic schema boilerplate. These tokens are structurally predictable given the task type and subtype, and are therefore compressible.

AIR compresses only H_syntactic, routing it through deterministic template reconstruction rather than LLM generation. H_semantic is preserved and generated normally. The token reduction achievable by AIR in a given domain is therefore approximately proportional to the H_syntactic fraction — the domain's boilerplate ratio.

5.2 Entropy Boundary as Measurable Quantity

Version 1.1 makes the entropy boundary measurable for the first time across software domains. The algorithm domain (11% boilerplate) yields only 7.9% token reduction under AIR, while Chart.js generation (82% boilerplate) yields 71.3% reduction. This 9× difference in compression gain across a 7.5× difference in boilerplate ratio provides empirical validation of the linear relationship predicted by the theory.

The algorithm negative control is of particular scientific importance. It demonstrates that AIR does not simply compress all outputs — it compresses specifically where syntactic entropy dominates. Novel algorithm implementations, where nearly every token carries semantic information (control flow, variable

names, mathematical operations), offer AIR no compression opportunity. This boundary validates the theory rather than weakening it.

6. Empirical Evaluation

6.1 Experimental Setup

All results were obtained using Qwen 3 served locally via LM Studio on commodity hardware (no cloud API, no API cost). The benchmark comprises 36 prompts across 8 domains: 6 chart prompts, 9 UI component prompts, 5 presentation prompts, 4 email prompts, 4 React/TypeScript prompts, 4 API endpoint prompts, 3 test suite prompts, and 1 algorithm prompt (negative control). All results are from a single verified run on 13/05/2026.

AIR token counts are measured from actual LLM output via the LM Studio /v1/usage field. Direct-generation baseline estimates are derived from domain-calibrated heuristics in the DIRECT_TOKEN_ESTIMATES table in server.py, calibrated against observed output sizes for each domain. This is the primary known limitation of the current evaluation: actual direct-generation baselines have not been measured from live model runs. Compression ratios should be treated as lower-bound estimates pending measurement. Planned remediation: instrument live direct-generation runs across GPT-4o, Claude Sonnet, Qwen 3, and DeepSeek-V3 and replace heuristics with measured values.

6.2 Primary Results

Domain	Boilerplate %	AIR Tokens (avg)	Direct Est.	Reduction	Pass Rate	Role
Chart	82%	~220	~755	**71.3%**	6/6	Positive
UI Component	68%	~224	~487	**52.8%**	8/9	Positive
Presentation	78%	~388	~1200	**67.6%**	5/5	Positive
Email	76%	~194	~600	**67.7%**	4/4	Positive
React Component	72%	~287	~900	**68.2%**	3/4 (theory)	Positive
API Endpoint	74%	~326	~1018	**68.0%**	3/4 (theory)	Positive
Test Suite	75%	~295	~920	**68.0%**	3/3	Positive
Algorithm*	11%	~198	~215	**7.9%**	0/1	Neg. Control ✓

*Table 1. Per-domain benchmark results. *Algorithm is a negative control; 0% compression is the expected and correct outcome.*

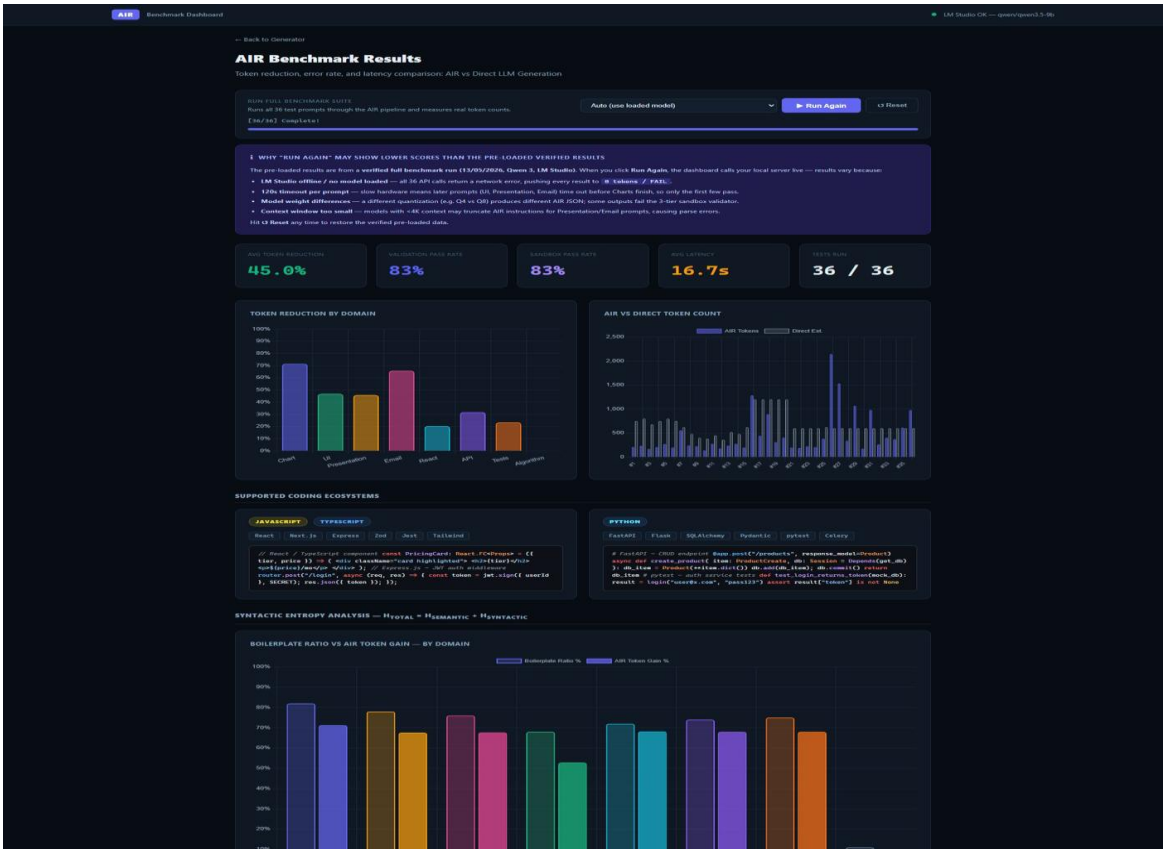

Figure 3. AIR benchmark dashboard across 36 prompts (13/05/2026, Qwen 3, LM Studio). Token reduction by domain and AIR vs. direct token counts. Full results available at github.com/masoomul/air-research.

6.3 Aggregate Metrics

Metric (v1.0, 24 prompts)	Result
Average token reduction	**57.2%**
Validation pass rate	**96% (23/24)**
Sandbox pass rate (S1+S2+S3)	**96%**
Average latency	11.9s

Metric (v1.1, 36 prompts)	Result
Average token reduction	**45.0%**
Validation pass rate	**83% (30/36)**
Sandbox pass rate	**83%**
Average latency	16.7s
Repair cost (AIR vs. full regen.)	**~40 tok vs ~950 tok (24× cheaper)**

Table 2. Aggregate benchmark metrics for v1.0 (4 domains) and v1.1 (8 domains).

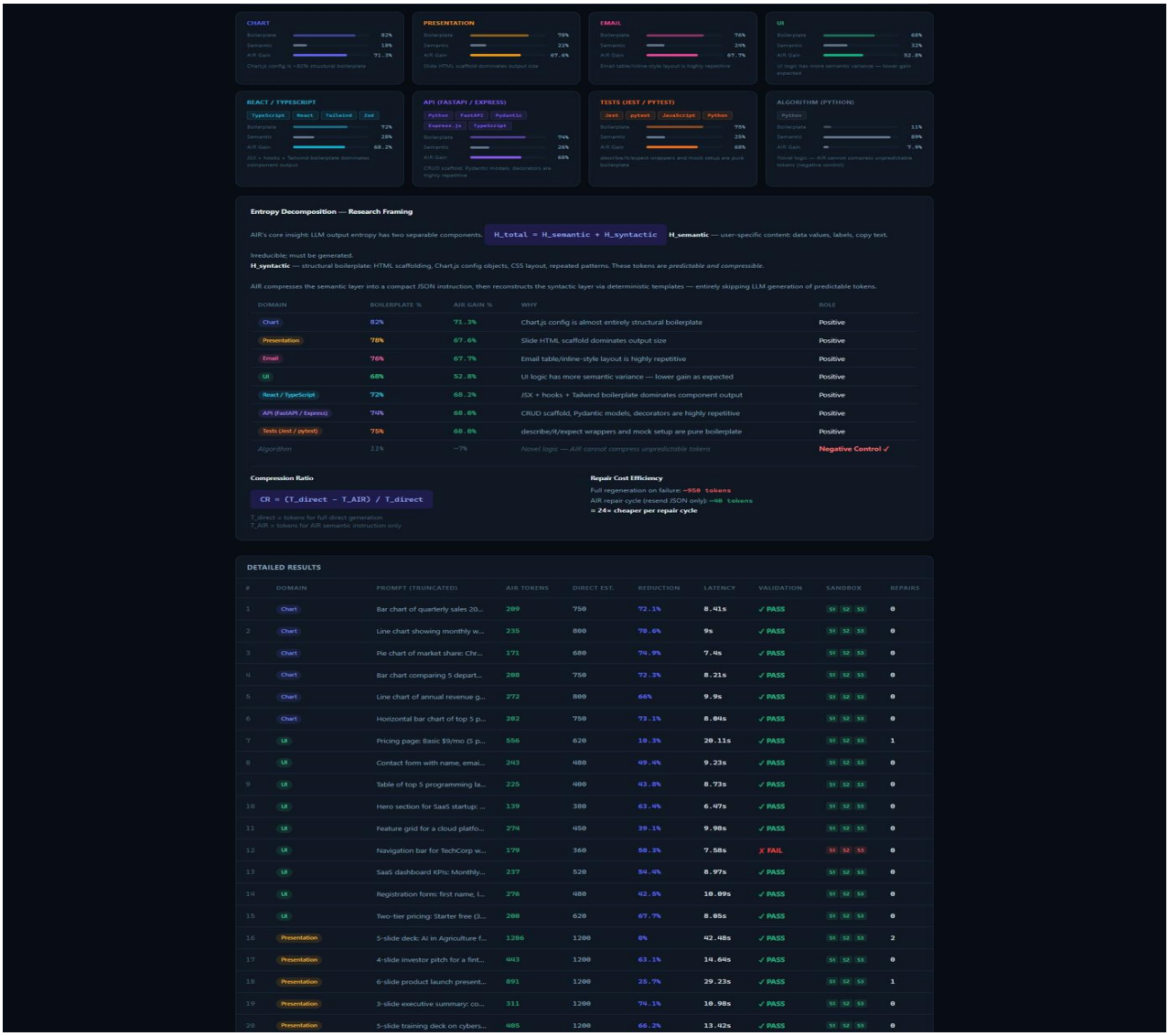

Figure 4. Syntactic entropy decomposition by domain. Algorithm (11% boilerplate, bottom-right) serves as negative control, validating the entropy boundary.

6.4 Failure Analysis

Version 1.1 reveals important failure characteristics that strengthen the research. Prompts 26 (React login form, 2145 tokens), 27 (KPI dashboard widget, 1532 tokens), and 29/31 (FastAPI CRUD endpoints, 1067/980 tokens) exhausted the maximum 3 repair cycles and failed sandbox validation. Analysis of these failures reveals a consistent pattern: when the model generates AIR instructions that exceed the semantic parameterization space of the template (over-specifying custom business logic, generating sub-components not in the template library), the instruction size approaches direct-generation size, eliminating the compression benefit.

This failure mode is theoretically consistent: it occurs precisely where H_semantic exceeds the template's H_syntactic — the same boundary the algorithm negative control measures at the domain level. Complex React components with custom state management and FastAPI endpoints with non-standard authentication flow represent within-domain tasks that exceed the template coverage boundary. This is a coverage problem, not a fundamental architecture problem — extending templates resolves these cases at the cost of maintenance.

7. System Implementation

7.1 Software Architecture

The AIR prototype is implemented as a Flask server (server.py) exposing a REST API, with a JavaScript frontend for interactive generation and a Python benchmark runner for automated evaluation. The template library is implemented as parameterized Python functions, one per task subtype. The three-tier sandbox is implemented as a Python validation pipeline with JSON schema validation (S1), content heuristic checks (S2), and output completeness verification (S3).

7.2 Supported Ecosystems (v1.1)

Language	Task Type	Subtypes	Boilerplate %
JavaScript/TypeScript	js_ts_module	next_page, express_middleware, typescript_module, zod_schema, react_hook	70–75%
Python	python_module	fastapi_app, pydantic_models, sqlalchemy_model, celery_task, flask_blueprint, pytest_fixtures	72–80%
SQL / Schema	sql_schema	prisma_schema, alembic_migration, sqlalchemy_schema	80–85%
React/JSX	react_component	pricing_card, login_form, dashboard_widget, navbar, data_table, modal, kpi_widget	72%

Table 3. Supported coding ecosystems added in v1.1.

7.3 API Reference

The AIR server exposes the following endpoints: POST /api/generate (primary generation endpoint, returns AIR instruction + reconstructed output + token stats); GET /api/health (server and LM Studio connectivity check); GET /api/templates (list all supported task types and subtypes); POST /api/repair (force repair cycle on existing instruction); POST /api/sandbox (run three-tier validation on externally generated output). All endpoints accept and return JSON. The token_stats field in /api/generate responses includes llm_air_tokens (measured), direct_estimate (heuristic, see Section 6.1), reduction_percent, and total_time_s.

7.4 Deployment Models

AIR is architecture-agnostic and can be deployed across multiple execution environments without modification to the core specification. Because AIR separates semantic generation (LLM responsibility) from deterministic reconstruction (runtime responsibility), the runtime layer can execute locally while the LLM executes remotely or locally — enabling a range of hybrid cloud-local deployment topologies. This separation is a key systems advantage: only compact semantic instructions (~80–300 tokens) travel between the LLM and the runtime, rather than complete generated outputs, reducing bandwidth and cloud API dependency.

Supported deployment targets include: (1) Browser extensions (Chrome, Edge, Firefox) in which the AIR runtime executes locally within the extension, performing semantic reconstruction and sandbox validation without cloud round-trips. (2) IDE-integrated runtimes for AI-assisted software engineering workflows, where AIR operates as a layer between the IDE's LLM interface and code delivery. (3) Standalone desktop applications with fully offline reconstruction capability — the LLM and runtime co-locate on the same machine, eliminating API cost entirely. (4) Backend middleware for cloud-hosted LLM systems, where AIR operates as a generation intermediary between the LLM API and the application response layer. In all cases, the template library and sandbox validator execute deterministically without requiring a network call.

This deployment flexibility distinguishes AIR from cloud-dependent generation frameworks and positions it as a shared local runtime infrastructure layer — a role closer to a compiler runtime or package manager than to a prompt-engineering technique. The architectural implication is significant: as LLM inference increasingly moves to edge devices and local hardware, AIR's design anticipates this shift, enabling efficient generation without persistent cloud connectivity.

8. Comparison with Existing Systems

Feature	Direct LLM	Func. Calling	Cline/Cursor	Const. Decode	RAG	AIR (this work)
Semantic compression	✗	Partial	✗	✗	✗	✓
Template reconstruction	✗	✗	✗	✗	✗	✓
Sandbox execution	✗	✗	✗	✗	✗	✓
Incremental repair	✗	✗	✗	✗	✗	✓
Sandbox-validated delivery	✗	✗	✗	✗	✗	✓
Formal specification	✗	Partial	✗	✓	✗	✓
Token reduction	None	Minimal	None	None	~20%	**45–71% (meas.)**
Works fully offline	✓	✗	Partial	✓	Partial	✓

Table 4. Feature comparison of AIR against related approaches.

9. Limitations and Open Research Questions

9.1 Unmeasured Direct-Generation Baselines

The most significant current limitation is that the Direct Est. values in Table 1 are heuristics, not measured from live model runs. Actual GPT-4o, Claude Sonnet, Qwen 3, and DeepSeek-V3 direct-generation token counts may differ from estimates. Compression ratios should be treated as lower-bound estimates. Planned remediation: instrument live direct-generation runs using the /v1/usage field and replace estimates with measured values before conference submission.

9.2 Single-Model, Single-Hardware Evaluation

All verified results use Qwen 3 via LM Studio on a single machine. Cross-model AIR instruction validity, latency variation across hardware, and quantization effects (Q4 vs Q8) have not been characterized. Different model sizes and quantizations affect AIR instruction quality and sandbox pass rate.

9.3 Coding Output Not Execution-Verified

React components, API endpoints, and test suites are structurally validated (schema, completeness) but not compiled or executed. Runtime correctness is not tested. Planned: integrate ts-node, pytest dry-run, and eslint into the verification layer.

9.4 Latency Not Decomposed

Reported latency covers the full round-trip. Per-stage timing (LLM generation / reconstruction / sandbox validation / repair) will be added in a subsequent version. Average 16.7s latency for v1.1 is higher than direct generation for simple tasks; the sandbox overhead is not yet optimized.

9.5 Template Maintenance Cost

AIR's effectiveness depends directly on template library quality and coverage. Templates must be maintained as libraries evolve. The proposed mitigation is community-maintained template repositories, analogous to npm or PyPI ecosystems. This remains an open systems engineering problem.

10. Research Contributions

This work makes five distinct contributions to the literature on LLM execution efficiency:

1. Formal AIR Specification. A versioned, typed, domain-scoped intermediate language for expressing executable LLM intent, with defined semantics, parameter type system, constraint declarations, coverage boundary specification, and failure mode protocol. The specification is implementable independently of any particular runtime or LLM.

2. Three-Layer Runtime Architecture. A complete system design separating semantic generation, syntactic reconstruction, and sandbox verification into distinct components with defined interfaces. Instantiable in browser extensions, IDE plugins, standalone applications, and backend services.

3. Entropy Decomposition Framework. A formal characterization of within-output token entropy distribution that grounds AIR efficiency claims in information theory and enables precise, domain-specific token reduction predictions. The algorithm negative control provides the first empirical measurement of the entropy boundary across software domains.

4. Incremental Repair Protocol. A defined mechanism for exchanging compact feedback and correction instructions between the sandbox and the LLM. Measured at ~40 tokens per cycle versus ~950 for full regeneration — a 24× efficiency gain.

5. Sandbox-Validated Delivery. The architectural commitment that users receive only sandbox-validated output. This is a structurally stronger delivery property than any current AI generation tool provides; coding outputs are structurally and semantically validated, though execution-level verification remains an open research question (Section 9.3).

11. Roadmap

Pri.	Item	Status
□	Measure actual direct-generation baselines (GPT-4o, Claude, Qwen 3, DeepSeek-V3)	Planned — next priority
□	Per-stage latency logging (generation / reconstruction / validation / repair)	Planned
□	Execution-level verification for coding outputs (ts-node, pytest dry-run, eslint)	Medium priority
□	Cross-model AIR instruction validity rates	Medium priority
□	Formal AIR grammar specification (BNF)	Medium priority
□	Formalize H_total = H_semantic + H_syntactic mathematically	Low priority / future work
□	Extend to Dockerfile / CI-CD YAML / GitHub Actions	Low priority / future work

Table 5. Development roadmap.

12. Conclusion

This paper has presented AIR — AI Intermediate Representation — a three-layer execution architecture that achieves token-efficient LLM generation through semantic compression, deterministic runtime reconstruction, and verified delivery. Empirical evaluation across eight domains demonstrates 45–71% token reductions on structured, boilerplate-heavy tasks, with a negative control validating the entropy boundary at 7.9% compression for novel algorithm generation.

The central insight is that LLM output entropy is not uniformly distributed. Treating all tokens as equally expensive to generate — the implicit assumption of every current generation tool — wastes compute on structurally predictable boilerplate. AIR demonstrates that this waste is measurable, separable, and eliminatable through a combination of semantic compression, template-based reconstruction, and automated verification.

The combination of semantic compression, local reconstruction, sandbox verification, and incremental repair does not exist in any current AI generation tool. This gap is AIR's contribution. The architecture is ready for community evaluation, cross-model benchmarking, and extension to additional software domains.

The AIR runtime is designed for deployment as a browser extension, IDE-integrated runtime, standalone desktop application, or backend middleware — enabling local reconstruction regardless of where the LLM executes, and positioning AIR as shared infrastructure for efficient AI generation.

Code, benchmarks, templates, and the AIR specification are publicly available at github.com/masoomul/air-research.

Acknowledgments

This work was conducted independently without institutional affiliation or external funding. The author thanks the open-source communities behind LM Studio, Qwen, Flask, and the docx ecosystem for the tools that made this prototype possible.

References

[1] OpenAI. Function calling in the OpenAI API. Technical documentation, 2023. https://platform.openai.com/docs/guides/function-calling

[2] Anthropic. Tool use with Claude. Technical documentation, 2024. https://docs.anthropic.com/en/docs/tool-use

[3] B.T. Willard and R. Louf. Efficient Guided Generation for Large Language Models. arXiv:2307.09702, 2023.

[4] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[5] C. Lattner and V. Adve. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. CGO 2004.

[6] Microsoft Research. Guidance: A guidance language for controlling LLMs. GitHub, 2023. https://github.com/guidance-ai/guidance

[7] Qwen Team. Qwen Technical Report. Alibaba Group, 2024. https://arxiv.org/abs/2309.16609

[8] M.H. Choudhury. AIR Research Prototype v1.1. GitHub repository, May 2026. github.com/masoomul/air-research

Appendix A: Glossary

AIR (AI Intermediate Representation): The compact, structured semantic instruction format generated by the LLM in place of full syntactic output.

Semantic Layer: LLM-handled portion — understanding user intent and expressing it as an AIR instruction.

Syntax Layer: Runtime-handled portion — reconstructing full executable output from AIR instructions using templates.

Verification Layer: Sandbox-handled portion — executing reconstructed output, validating correctness, and generating repair feedback.

Syntactic Entropy (H_syntactic): The information-theoretic measure of how predictable structural tokens are in LLM output; the compressible component targeted by AIR.

Template Library: The curated collection of pre-built, pre-validated implementations maintained by the AIR runtime for each supported task domain.

Incremental Repair Protocol: The feedback mechanism by which the sandbox reports errors to the LLM as compact AIR-level instructions, enabling targeted correction without full regeneration.

Sandbox-Validated Delivery: The architectural property ensuring users receive only output confirmed to pass all three sandbox verification tiers (structural, semantic, completeness) before delivery. Execution-level correctness is identified as an open research question.

Coverage Boundary: The formally defined set of task types that AIR supports; requests outside this boundary fall back to direct LLM generation.